

```
Row 1 palindrome? False
Row 2 palindrome? True
Row 3 palindrome? False
Row 4 palindrome? True
Row 5 palindrome? False
Row 6 palindrome? True
Row 7 palindrome? False
Row 8 palindrome? True
```

9. Du “vet” att det i en fil `almost_lowercase.txt` som ska bestå av endast gemener och tecken gömmer sig *ett enda* versalt ord, men du vet inte vilket det är. Det här tycker du är extremt jobbigt! Använd dig av `open` tillsammans med strängfunktioner för att *utan en loop* ta reda på vilket ord det är. Du kan anta att ett “ord” avgränsas av mellanrum och att alla övriga ord (utöver det du letar efter) består endast av gemener och/eller tecken.

Tips: någon av `strip`-funktionerna tillsammans med indexering kan komma till användning här.

Exempel på `almost_lowercase.txt`:

```
this is a file that /should/ consist of only lowercase letters.
but it doesn't!
there is a hidden upppercase word right HERE that you are supposed to find.
```

Kapitel 7

Moduler, bibliotek och programstruktur

7.1 Moduler

Många programmeringsspråk har ett “modulsystem”, så även Python. Dessa är smidiga när man skriver större program och egna bibliotek. De kan användas för:

- Kodåtervinning – importera kod som andra programmerare (eller du själv) har skrivit.
- Struktur – det är svårt att arbeta med mycket kod i en fil, så sprid istället ut den över flera filer.
- Samarbete – det är mycket lättare att skriva kod tillsammans som ett team om den är uppdaterad i flera filer så att alla inte behöver ändra i samma fil.

Med hjälp av moduler kan du börja förstå ett stort projekt som bestående av självständiga enheter. Du ska inte behöva förstå alla detaljer i en modul, det ska räcka att förstå dess *gränssnitt*. Gränssnittet är de funktioner och variabler som är tänkta att vara synliga för andra programmare som använder sig av modulen.

Stora program blir snabbt överskådliga. Väl valda moduler kan då bli viktiga för att bättre förstå ditt och andras program.

En teknisk fördel med moduler är att de hjälper dig att hantera *namnrymden* (eng. *name space*). Många kan vilja använda samma identifierare för t.ex. funktioner i ett stort program (exempelvis `sort`). Moduler döljer implementationen och variabel-/funktionsnamn.

Principer:

- Varje Python-fil är en modul.
- Allt som ska användas (från en annan fil) måste *importeras*.

7.1.1 Import av moduler

Man kan importera moduler på ett antal olika sätt:

- **import** modulename: Tillåter att funktioner/variabler används med punktnotation (dvs allt från modulename är importerat “qualified”).

```
import math
math.ceil(3.5) # Avrunda uppåt
ceil(4.4) # Fel!
```

- `from modulename import f1, f2, ...`: Tillåter användning av `f1` och `f2` i ditt program utan punktnotation.

```
from math import ceil
ceil(3.5)
```

- `from modulename import *`: Gör allt från `modulename` tillgängligt utan punktnotation.

Varning: detta anses orsaka dålig läsbarhet och det blir även svårt att veta vilka delar av en modul en annan modul beror på.

Obs: identifierare som börjar med `_` importeras inte. De anses "privata" till modulen.

Notera: vissa moduler bygger på att man `*-importerar`, då de *behöver* (eller är skrivna så att) skriva över inbyggda funktioner i Python för att fungera.

- `import modulename as X`: Lättare hantera långa modulnamn när man använder punktnotation. Exempel:

```
import math as M
M.ceil(3.4) # Praktiskt för långa modulnamn
```

- `from modulename import X as Y`: För att undvika namnkrockar. Exempel:

```
from math import gcd as libgcd

def gcd(a,b):
    r = a % b
    if r == 0:
        return b
    else:
        return gcd(b, r)

for a in range(2,10):
    for b in range(2,a):
        print(gcd(a,b) == libgcd(a,b))
```

- Moduler kan grupperas i *paket* (eng. *packages*), vilka ska förstås som hierarkiskt ordnade grupper av moduler.

```
import os.path as OS
if OS.isfile('data.txt'):
    ...
```

Vilket sätt att importera moduler ska man då använda? Det beror förstås på omständigheterna. Ett praktiskt exempel finns här nedan, där en funktion för att beräkna binomialkoefficienter definieras på fyra olika sätt. Man brukar dock rekommendera att inte använda `from m import *` eftersom risken för namnkrockar ökar och det blir otydligt vad du använder från den importerade modulen.

(a)

```
import math
def binom1(n, k):
    return math.factorial(n) / (math.factorial(k) * math.factorial(n-k))
```

(b)

```
import math as m
def binom2(n, k):
    return m.factorial(n) / (m.factorial(k) * m.factorial(n-k))
```

(c)

```
from math import factorial
def binom3(n, k):
    return factorial(n) / (factorial(k)*factorial(n-k))
```

(d)

```
from math import factorial as fac
def binom4(n, k):
    return fac(n) / (fac(k)*fac(n-k))
```

Vilket alternativ av (a) till (d) är mest lättläst, tycker du? Tips: modulen `math` har en funktion `comb` som är snabbare än den som presenteras här.

7.1.2 Skriva egna moduler

Låt oss säga att vi har följande Python kod i en fil `circle.py`:

```
pi = 3.14

def area(radius):
    return pi * (radius ** 2)

def circumference(radius):
    return 2 * pi * radius
```

Vi kan då importera den i en annan fil:

```
import circle

print(circle.pi)
print(circle.area(2))

import circle as C

print(C.pi == circle.pi)

tau = C.pi * 2

def circumference(radius):
    return tau * radius
```

```
print(C.circumference(2) == circumference(2))
```

Notera 1: filen `circle.py` måste hittas av Python när programmet letar efter potentiella moduler att importera. Den första platsen Python letar på är i samma mapp som det `program/.py`-fil man kör ligger i; det enklaste sättet är därför att ha sin modulfil (här `circle.py`) i *samma* mapp som huvudprogrammet/filen som importerar modulen.

Notera 2: för att använda ett värde på π så importerar man normalt `math.pi`, som kommer med bra precision.

```
from math import pi
print(pi)
```

```
3.141592653589793
```

7.1.3 `main()`-funktioner

Ett vanligt idiom:

```
print("Hello")

def main():
    print("python main function")

if __name__ == '__main__':
    main()

print("__name__ value: ", __name__)
```

Om och endast om man startar programmet med denna fil så anropas `main`. Om man använder filen som en modul så körs inte `main`. Du kan alltså skriva moduler som innehåller exempelprogram och/eller testkod som inte påverkar vad som händer vid `import`. Anledningen att `main` inte körs om man importerar filen som en modul är att variabeln `__name__` sätts till modulnamnet, så `if`-satsen kommer då inte att köras.

7.2 Populära bibliotek och moduler

Python kommer laddat med batterier: det finns massor av bra och användbara moduler som är en del av Python-implementationen. Dessa moduler kallas tillsammans Pythons standardbibliotek. För en lång lista samt dokumentation se: <https://docs.python.org/3/library/>.

- `math` – Matematik: trigonometri, logaritmer, osv.
- `sys` – Avancerad filhantering, med mera...
- `os` – Undvik att vara beroende av Win/Mac/Linux (typ undvik problem med `"/` i filnamn i Linux/Mac vs. `"\"` i filnamn för Windows).
- `argparse` – Används för parametrar när man startar program på kommandoraden.
- `itertools` – Bra för avancerade och snabba iterationer.
- `functools` – Fler högre ordningens funktioner.

- `datetime` – Datum och tidstyper.
- `pickle` – Serialisering av Python objekt.
- `csv` – Det är svårare än du tror att läsa en komma-separerad fil!
- `json` – Spara data i JSON-format för att lätt kunna läsa in igen.
- `matplotlib` – Visualisera data.
- `numpy` och `scipy` – För beräkningar.
- ...: Utforska själv!

7.2.1 JSON och serialisering

Problem: Jobbigt att skriva och läsa filer! Särskilt med stora datastrukturer (tänk: nästlade listor och/eller uppslagstabeller).

Lösning: *serialisering*

Idé: Dumpa datastruktur på fil i standardformat som lätt kan läsas in igen.

Finns standard pythonmoduler för detta (`marshal` och `pickle`).

Bra portabel lösning: JSON – JavaScript Object Notation

```
import json
legends = [ ['Lovelace', 'Ada', 1815]
            , ['Babbage', 'Charles', 1791]
            , ['Beurling', 'Arne', 1905]]

# Skapa datafil:
with open('legends.json', 'w') as outfile:
    outfile.write(json.dumps(legends))

# Senare... Skriv ut födelsesår:
with open('legends.json', 'r') as data_h:
    external_data = json.load(data_h)
    for x in external_data:
        print(x[2])                # Print birth year
```

7.2.2 NumPy: för vektorer och matriser

Med `numpy` (<https://www.numpy.org/>) får du ett paket för vetenskapliga beräkningar i Python. Det kommer med mycket effektiv lagring av vektorer och matriser, m.h.a. “arrays”. Pythons list har gränssnitt som (bl.a.) en array, men är mycket mer flexibelt. Det kostar dock i effektivitet. Arrayer i `numpy` är närmare datorns egen representation, vilket gör beräkningar mer effektiva (bättre användning av minne och beräkningstid). Effektiviteten kommer på bekostnad av flexibilitet, men i många tillämpningar är det mer än väl värt.

Skapa vektorer och matriser med `numpy`-arrayer:

```
import numpy as np          # idiomatisk Python att importera numpy som np

a = np.array([1,2,3,4])    # Från vanlig lista
```

```

b = np.ones((2,2))      # 2x2-matris med ettor, notera tupeln!
c = np.zeros((10,10))   # 10x10-noll-matris
d = np.linspace(0, 1, 16) # 16 element från 0 till 1, flyttal!
e = np.reshape(d, (4,4)) # Ger 4x4-matris med tal från 0 till 1
f = np.identity(2)       # Enhetsmatris, 2x2

print(a)
print('Only ones:', b)
print('Large matrix of zeros:', c)
print(d)
print(e)
print('Unity matrix:', f)

```

```

[1 2 3 4]
Only ones: [[1. 1.]
 [1. 1.]]
Large matrix of zeros: [[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
[[0.         0.06666667 0.13333333 0.2         0.26666667 0.33333333
 0.4         0.46666667 0.53333333 0.6         0.66666667 0.73333333
 0.8         0.86666667 0.93333333 1.         ]
 [[0.         0.06666667 0.13333333 0.2         ]
 [0.26666667 0.33333333 0.4         0.46666667]
 [0.53333333 0.6         0.66666667 0.73333333]
 [0.8         0.86666667 0.93333333 1.         ]]
Unity matrix: [[1. 0.]
 [0. 1.]]

```

Notera att np är ett etablerat alias, och import görs oftast som i exemplet, inklusive i numpy-dokumentationen.

Tips: prova exempelvis `help(np.linspace)` för att se hur den fungerar.

Man har naturligtvis tillgång till operationer på matriser, som t.ex. matrismultiplikation:

```

m = np.array([[1,2], [3,4]])
v = np.array([1, 0.5])
product = np.matmul(m, v)

```

```

[[2. 5.]]

```

numpy betraktar generöst en array som en vektor på ett sätt som passar in. Här ovan tolkas alltså `v` som en kolonnvektor, och om man istället skriver `np.matmul(v, m)` tolkas den som radvektor.

Det finns en särskild datastruktur för matriser definierad i numpy, men den bör man idag undvika eftersom den planeras att plockas bort: rekommendationen är att alltid använda `np.array`.

numpy är en del av scipy (<https://www.scipy.org/>) som innehåller en massa olika paket för att göra vetenskapliga beräkningar och hantera “stor” data med Python.

7.3 Tumregler för programstruktur och bra kod

Det är svårt att skriva bra kod. Med “bra” kod menas kod som är lätt att läsa, förstå sig på, använda och underhålla. Genom att skriva bra kod kan vi även undvika att introducera buggar. Det är dock svårt att säga exakt vad som gör kod bra, men förhoppningsvis har ni fått se lite exempel på både bra och mindre bra kod under kursens gång. Många programmerare har olika definitioner på vad som gör kod bra (på samma sätt som om det är väldigt subjektivt vad som är “bra konst” eller “bra litteratur”).

Man kan hitta en massa specifika Python-tips på: <https://docs.python-guide.org/writing/style/>. Det finns en stilguide, med det stela namnet [PEP-8](#), som är skriven av Guido van Rossum, skaparen av Python. När du börjar programmera i andra språk är det en bra idé att se vilka principer som gäller där.

Det finns verktyg för automatisk granskning av kod. Ett sådant finns på nätet: <https://www.pythonchecker.com/>. Det finns också IDE:er med liknande stöd, och bibliotek som `pylint` som man kan köra i ett terminalfönster.

7.3.1 Programmeringsriktlinjer

Här kommer en lista (i ingen specifik ordning) över saker som vi tycker gör kod bättre:

- Undvik långa rader, mindre än 79 tecken per rad är bra. Om en rad är för lång är det väldigt svårt att läsa den. [PEP-8](#) kräver denna begränsning.
- En kodsnuitt per rad. Följande kod är dålig:

```
print('one'); print('two')

if x == 1: print('one')

if <complex comparison> and <other complex comparison>:
    # do something
```

Följande kod är mycket bättre:

```
print('one')
print('two')

if x == 1:
    print('one')

cond1 = <complex comparison>
cond2 = <other complex comparison>
if cond1 and cond2:
    # do something
```

- Eftersträva att skriva korta funktioner. De flesta funktioner bör få plats på en halv laptopskärm med teckenstorlek 12. I vissa fall kan de självklart bli längre, men tänk då noga på om det går (eller

blir tydligare) att faktorisera ut koden i mindre funktioner först. Om funktioner är väldigt långa är det svårt att få överblick och hålla reda på allt funktionen gör.

- Dela upp kod i funktioner som gör **en** sak och gör det bra. Detta gör det mycket lättare att läsa, testa och förstå koden.

Exempel: “läs in flera datamängder, gör beräkningar på varje datamängd, skriv ut en sammanfattning” bör delas upp på följande sätt:

- En funktion för inläsning av datamängder.
 - En funktion som gör beräkningen på en datamängd.
 - En huvudfunktion som anropar de två andra funktionerna och skriver ut sammanfattningen.
- Funktioner bör returnera något. Det blir då möjligt att testa dom och det är lättare att förstå vad de gör eller förväntas göra. Det finns dock tillfällen då en funktion inte behöver returnera något (utskrifter, listmanipulation, osv), men förväntningen bör ändå vara att funktioner returnerar något. Ett vanligt nybörjarfel är att man använder globala variabler för att hantera dataflödet i sitt program och då “behövs inte” returvärdet. För nybörjaren kan alltså brist på returvärdet vara en indikation på dålig programstruktur.
 - Varje funktion ska ha en dokumentationssträng.
 - Separera **beräkning** från **interaktion**: funktioner som gör beräkningar bör inte ha print/input. Funktioner som hanterar interaktion bör inte göra komplexa beräkningar. Interaktion är såväl enkla utskrifter som frågor till användaren. Man vill ofta återanvända beräknande funktioner, men inte deras diagnostiska utskrifter.
 - Tänk först på hur programmet bör struktureras och vilka abstraktioner som bör användas innan ni börjar koda. Gruppera funktionalitet och attribut i klasser¹—på så sätt uppnås abstraktion och det blir lättare att ändra i programmet senare.
 - Dela upp i små enheter, d.v.s. dela upp i lämpliga funktioner och klasser. Detta kallas för *modulär* kod då varje logisk del i koden kan studeras som en egen liten modul som sedan kombineras med andra moduler för att skapa komplexa program. Det gör programmet mycket mer överskådligt och lättförståeligt.
 - Returnera **inte** värden av typ `int` eller `bool` som strängar, d.v.s. som `'2'` eller `'True'`. Det gör det svårare att testa och använda funktionerna senare. Det är även lätt att introducera buggar på så sätt, exempelvis:

```
def f(x):  
    return '2'  
  
if f(1) == 2:  
    print("This is not printed!")  
else:  
    print("This is printed!")
```

- Returnera inte “magiska” värden för att markera fel eller problem. Använd hellre särfall. Exempelvis: “return 1000000” är dålig markör för “konvergerade inte”. **Undantag**: None kan vara ett acceptabelt magiskt värde.
- Multipla returvärdet är praktiska! Bättre än strängar som innehåller flera värden.

¹Klasser introduceras i kapitel 9.

- Variabler/konstanter är bättre än hårdkodade litteraler. Till exempel om vi skriver ett grafiskt program och vill ha ett fönster med storlek 600 x 600, då är det bättre att först skriva

```
width , height = 600 , 600
```

och sen använda width/height istället för att upprepa 600 överallt. På så sätt blir det lättare att ändra programmet i framtiden om vi vill ha andra dimensioner på fönstret. Det blir också lättare att läsa koden om det står width istället för ett godtyckligt nummer.

- Undvik globala variabler då de ofta leder till “spagettikod” (d.v.s. rörig kod där vitt skilda delar av programmet kan påverka varandra). Man ska kunna förstå en funktion utan att titta på andra delar av ett program. Det är direkt olämpligt att manipulera globala variabler i funktioner (d.v.s. använda nyckelordet **global**).
- Inkludera tester för att både testa och specificera hur program är tänkta att fungera. Detta är lättare att göra om programmet är uppdelat i mindre funktioner som gör **en** sak och **returnerar** den.
- Enkel kod är lättare att läsa. Undvik därför att skriva onödigt komplicerade kodsnuttar. Till exempel är “**if** b == **True**.” mer komplicerat än bara “**if** b:”. Det är som med vanlig svenska: det är dålig stil att stapla många små ord på varandra. På samma sätt som enkla meningar blir lättare att förstå, så är enkel kod lättare att förstå.

Här är ett exempel på ett onödigt komplicerat program:

```
def make_complex(args):
    x, y = args
    d = dict()
    d['x'] = x
    d['y'] = y
    return d
```

Mycket bättre:

```
def make_complex(x,y):
    return {'x': x, 'y': y}
```

Ett bra knep är att alltid läsa igenom sin kod flera gånger och reflektera över hur den kan förenklas.

- Var försiktiga med muterbara datatyper (många buggar uppstår då listor råkas tömmas genom exempelvis `pop()`!). Speciellt farligt är det att ha muterbara typer som globala variabler eller klassattribut.
- Dela upp större projekt i logiska enheter i olika filer/mappar.
- Det ska framgå av identifieraren vad en variabel lagrar eller vad en funktion är till för. Som exempel, om du har raden

```
x = 17 # critical value, abort if exceeding
```

så är det nog bättre att byta x mot något annat, kanske så här:

```
critical_value = 17 # abort if exceeding
```

Ett till exempel: istället för

```
nc = 1.7 # normalization constant
```

bör man skriva:

```
normalization_constant = 1.7
```

Här behövs det då ingen kommentar.

- Det finns tillfällen när funktionsnamnen får vara oinformativa: t.ex. `main`, pga konventioner
- Undvik generiska identifierare. Använd identifierare som är beskrivande.
 - *Dåliga*: `start`, `compute`, `f`, `foo`
 - *Bra*: `remove_outliers`, `newton_raphson`, `compute_integral`
- Kod ändras. Om funktionens syfte ändras så bör du byta identifierare! Om din funktion heter `print_integral` men inte gör några utskrifter så är det förvirrande.
- Kommentarer är bra, men ska inte ersätta enkel kod med bra namn på identifierare. Om koden behöver kommenteras för att vara förståelig så betyder det troligtvis att man gör något komplicerat. Bra användning av kommentarer är att förklara varför man använder en specifik datastruktur (t.ex. varför man använder ett set istället för `list`, etc.) eller hur det är tänkt att en funktion ska användas.

Kommentarer som inte bidrar något mer än vad koden redan säger bör undvikas. Till exempel är följande en helt redundant kommentar:

```
rows = [] # Create an empty list
```

Alla som kan lite Python förstår ju att kodraden skapar en tom lista, så varför skriva det i en kommentar?

7.3.2 Tecken på mindre bra kod

- “Copy-paste-kod”: det är lätt att introducera buggar om man kopierar rader och klistrar in dem igen. Programmet blir även svårsläst om det blir väldigt långt. Kan man introducera en loop eller funktion istället?
- Är det svårt att skapa bra tester? Då har du nog inte delat upp ditt program i små delar som gör **en** specifik sak. Tänk över designen och introducera bättre funktioner och abstraktioner.
- Djup indentering: om man har väldigt nästlade **if**-satser så är koden svårsläst och bör delas upp i mindre delar. En vanlig orsak till djup indentering är att man har flera olika logiska fall—skapa då funktioner för de olika fallen. På så vis blir fallanalysen tydligare och de olika funktionsanropen indikerar vad som sker i fallen.

Här är ett exempel på “dålig” kod:

```
# Print the prime numbers from a collatz sequence
# for 100 consecutive starting numbers
for i in range(1,100):
    n = i
    primes = []
    while n != 1:
        is_p = True
        for x in range(2, n):
            if not (n % x):
                is_p = False
```

```

        if is_p:
            primes.append(n)
        if n % 2 == 0:
            n = n//2
        else:
            n = 3*n + 1
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Detta kodblock är indenterat i fyra nivåer. Det är inte så farligt, men man kan strukturera det bättre:

```

def collatz_primes(n):
    """Computes prime numbers from
    the collatz sequence for an integer n.
    Parameters:
    n (int): input number
    Returns:
    primes: list of prime numbers in collatz sequence for n
    """
    primes = []
    while n != 1:
        is_p = True
        for x in range(2, n):
            if not (n % x):
                is_p = False
        if is_p:
            primes.append(n)
        if n % 2 == 0:
            n = n//2
        else:
            n = 3*n + 1
    return primes

for i in range(1,100):
    primes = collatz_primes(i)
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Här har vi brutit ut en särskild funktion, `collatz_primes(n)`, som används i en `for`-loop, och den funktionen har bara tre indenteringsnivåer (`while`-slingan, `for`-slingan och `if`-satsen).

Vi kan ta det ett steg till och lägga uppdateringen av variabeln `n` och beräkningen om det är ett primtal i egna funktioner:

```

def collatz_update(n):
    if n % 2 == 0:
        return n//2
    else:
        return 3*n + 1

def is_prime(n):

```

```

if n < 2:
    return False
if n == 2:
    return True

for y in range(2,n):
    if not (n % y):
        return False
return True

def collatz_primes(n):
    """Computes prime numbers from
    the collatz sequence for an integer n.
    Parameters:
    n (int): input number
    Returns:
    list: list of prime numbers in collatz sequence for n
    """
    primes = []
    while n != 1:
        if is_prime(n):
            primes.append(n)
        n = collatz_update(n)
    return primes

for i in range(1,100):
    primes = collatz_primes(i)
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Vilken version tycker du är lättast att läsa och förstå?

- Saknas det parametrar och **return** i funktioner som ska göra beräkningar? Då gör de troligtvis en massa konstigheter. Funktioners syfte är vanligen att ta indata och beräkna något. Beräkningen behöver vara inte vara matematisk utan kan vara enkel behandling av data. Det är mindre vanligt att funktioner helt saknar parametrar, men det finns förstås undantag, till exempel för utskrift av en meny eller initialisering av datastrukturer. Hos nybörjare är det dock inte ovanligt att man finner det bekvämt att lägga data i en eller flera globala variabler som funktioner sedan arbetar med. En sån konstruktion gör det svårt att se hur funktioner beror på varandra; man måste läsa funktionerna noggrant för att identifiera programflödet. Man kan sen inte heller använda funktionen i en annan modul om den inte tar någon parameter.

Koden nedan fungerar om vi lägger `is_prime` i annan modul `m` (d.v.s. fil) och kallar på den med `m.is_prime(x)`.

```

def is_prime(x):
    if x >= 2:
        for y in range(2,x):
            if not (x % y):
                return False
    else:

```

```

        return False
    return True

def main(x):
    print("The number is prime:", is_prime(x))

x = int(input("Provide a number to analyze: \n"))
main(x)

```

Koden nedan är sämre skriven och fungerar endast om vi har den globala variabeln `x` i samma fil. Detta gör att vi inte kan importera funktionen i en annan fil.

```

def is_prime():
    if x >= 2:
        for y in range(2,x):
            if not (x % y):
                return False
    else:
        return False
    return True

def main(x):
    print("The number is prime:", is_prime())

x = int(input("Provide a number to analyze: \n"))
main(x)

```

7.4 Uppgifter

1. Importera biblioteket `random` (se <https://docs.python.org/3/library/random.html>) för att generera 10 slumpnässiga *heltal* mellan 1 och 100 och 10 slumpnässiga *flyttal* mellan 1 och 100.

Tips: Läs dokumentationen. Vilka funktioner är lämpligast att använda?

2. Betrakta koden nedan som läser in ett tal och avgör om det är ett perfekt tal eller ett primtal. Skriv om koden så att den får bättre struktur.

Tips: Dela upp koden i funktioner. Vilka bitar i koden tenderar att bli naturliga funktioner? Vad bör funktionerna ta för parametrar och returnera för att vara återanvändbara utanför modulen? Ha gärna en huvudfunktion `main(x)` som tar talet `x` som parameter.

```

# checks if a number is (1) perfect (2) prime
x = int(input("Provide a number to analyze: \n"))
sum = 0
for i in range(1, x):
    if (x % i == 0):
        sum = sum + i

if x >= 2:
    prime = True

```

```

    for y in range(2,x):
        if not (x % y):
            prime = False
    else:
        prime = False

    if sum == x:
        print("The number is perfect")
    else:
        print("The number is not perfect")
    if prime:
        print("The number is prime")
    else:
        print("The number is not prime")

```

3. Lägg till i ditt omstrukturerade program från uppgift 2 valet att låta användaren bestämma om hen ska ange ett tal eller om programmet ska slumpa fram ett tal. Var i programmet bör vi placera denna nya kod?
4. Nedan kod läser in en DNA-sträng (t.ex. 'AGCTAGCGGTAGC') och letar först upp den vanligast förekommande strängen av längd k. Sedan skriver programmet ut alla parvisa avstånd mellan delsträngen på vår DNA-sträng. Till exempel för k = 3 returnerar koden nedan [4, 6] då det är 4 respektive 6 nukleotider mellan startpositionerna för den vanligast förekommande delsekvensen 'AGC'.

Uppgift: Skriv om programmet så att det blir mer läsbart. Vilka bitar i koden utgör lämpliga funktioner? Se till så att de både tar lämpliga parametrar och returnerar lämpliga datastrukturer. Funktionerna ska kunna kallas på individuellt från annan modul.

```

# Given a sequence as input find distances between the copies of
# the most frequent substring

seq = input("Enter a DNA string: ")
k = 3
substrings = {}
for i in range(len(seq) - k + 1):
    substring = seq[i:i+k]
    if substring in substrings:
        substrings[substring] += 1
    else:
        substrings[substring] = 1

max_count = 0
most_freq_substring = ""
for substring, count in substrings.items():
    if count > max_count:
        most_freq_substring = substring
        max_count = count

```

```

positions = []
for i in range(len(seq) - k + 1):
    substring = seq[i: i+k]
    if substring == most_freq_substring:
        positions.append(i)

distances = []
for p1,p2 in zip(positions[:-1], positions[1:]):
    distances.append(p2 - p1)

print(distances)

```

5. † Du har fem nummer i en lista [13, 24, 42, 66, 78] och vill beräkna alla möjliga sätt ett nummer kan skrivas som en “direkt” sammanslagning av 2 element ur listan; med en “direkt sammanslagning” av två element (heltal) menas här t.ex. 13, 78 -> 1378 och 66, 42 -> 6642. Dvs, du vill ta reda på alla permutationer av två element ur listan så och slå samman varje permutation till ett tal.

Tips: Titta på standardmodulen `itertools` för permutationerna. För att slå samman heltal, försöka att först explicit typomvandla talen i listan till `str`, sen konkatenera permutationerna m.h.a strängfunktionen `.join()` och till sista typomvandla tillbaka till `int`.

Utdata bör bli (om alla tal sparats i en lista som man printar):

```
[1324, 1342, 1366, 1378, 2413, 2442, 2466, 2478, 4213, 4224, 4266, 4278, 6613,
6624, 6642, 6678, 7813, 7824, 7842, 7866]
```

6. Skriv om laboration 1 (temperaturkonvertering) så att funktionerna hamnar i separat modul med namnet `temp_functions.py` och dokumentera funktionerna med en dokumentationssträng (se kapitel 2). Huvudprogrammet ska importera funktionerna i `temp_functions.py` samt hantera fel med `try/except`.